LABORATORUL 4

Input și actualizarea stării

Transformăm tabla statică într-un prototip controlabil.

- Actualizăm pozițiile, alegem direcția și viteza și oprim lucrul la pauză.
- Aceasta este ultima temă pe Snake. La final alegem jocul pentru proiect.

2 Legătura cu cursul

Curs 2	evenimente UI și lifecycle
Curs 3, p. 5–8	procesul în care rulează codul
Curs 4, p. 8–9	consumul de resurse și restricțiile de fundal

- Handler și bucla sunt aplicații practice noi. Senzorul anticipează Cursul 6, p. 27; nu presupunem că a fost deja predat.

3 Un pas al jocului

Mișcarea rezultă din actualizarea stării și desenarea rezultatului.

```
public void updateGame() {
    state.update(COLS, ROWS);
    invalidate();
}
```

- updateGame() este în BoardView. update() din GameState aplică regulile și mută șarpele.
- invalidate() solicită redesenarea; nu apelează imediat onDraw().

onDraw() afișează ultima stare disponibilă.

4 Direcția ca deplasare

Direcțiile folosesc (dx,dy): dreapta (1,0), stânga (-1,0), sus (0,-1), jos (0,1).

```
int nextCol = head.col + dx;
int nextRow = head.row + dy;
```

- *Rândurile cresc în jos. Direcția nu este poziția capului și nu reprezintă un număr de pixeli.*

5 Actualizarea șarpelui

La lungime constantă, adăugăm un cap nou și eliminăm coada.

```
snake.add(0, new Cell(nextCol, nextRow));
snake.remove(snake.size() - 1);
```

- Lista este modificabilă. Primul element reprezintă capul; ultimul, coada.

Corpul ocupă pozițiile anterioare ale capului.

6 Regula marginii în prototip

- La acest laborator, șarpele reapare pe partea opusă.

```
int nextCol = (head.col + dx + cols) % cols;
int nextRow = (head.row + dy + rows) % rows;
```

- Pe 10 coloane: 9 + dreapta devine 0; 0 + stânga devine 9.
- Adăugarea dimensiunii evită rezultatul negativ pentru pasul de o celulă.
- cols și rows sunt parametrii lui update(); BoardView îi trimite COLS și ROWS.

Obstacolele rămân vizuale; coliziunile vin la Lab 5.

7 Controlul prin evenimente

Listener-ul cere o direcție; update() realizează deplasarea la următorul pas.

```
buttonUp.setOnClickListener(v ->
    boardView.changeDirection(0, -1));
```

- changeDirection() deleagă către state.requestDirection().

Respinge sensul opus și permite cel mult o schimbare acceptată între două actualizări.

8 Handler și Runnable

Handler programează operații pe coada firului asociat. Runnable descrie operația prin run().

```
Handler handler = new Handler(Looper.getMainLooper());
handler.postDelayed(tick, intervalMs);
```

- Looper.getMainLooper() selectează firul principal. postDelayed() programează o execuție; repetarea apare când tick se reprogramează.

Nu creăm un fir nou. Întârzierea nu este un ceas exact.

9

Intervalul și viteza

- În Snake avansăm o celulă pe pas.
- 500 ms: aproximativ 2 pași pe secundă.
- 200 ms: aproximativ 5 pași pe secundă.
- Un interval mai mic produce mișcare mai rapidă.
- setIntervalMs() este metoda noastră: schimbă valoarea folosită la următoarele programări, fără o buclă nouă.

În cod folosim 300 ms: gameLoop.setIntervalMs(300L).

10 Pornire și oprire fără duplicare

start() este idempotent: repetarea apelului nu adaugă un al doilea lanț.

```
if (running) return;
running = true;
handler.postDelayed(tick, intervalMs);
```

- stop() dezactivează și elimină execuțiile programate:

```
running = false;
handler.removeCallbacks(tick);
```

- *tick verifică running înainte de lucru și înainte de reprogramare.*

11 Pauza și revenirea

- În prototip, Activity coordonează bucla:

```
@Override protected void onResume() {
    super.onResume();
    gameLoop.start();
}
@Override protected void onPause() {
    gameLoop.stop();
    super.onPause();
}
```

- Oprirea buclei păstrează starea numai cât obiectele respective există. Nu asigură restaurarea după recreare sau terminarea procesului.

12

Verificarea în Logcat

Filtru: tag:Lab04. Logăm START, STOP, intervalul și contorul pașilor.

- Verificarea 1: schimbăm viteza și observăm diferența.
- Verificarea 2: Home, pauză, revenire; contorul nu avansează în pauză pentru aceeași instanță și jocul nu accelerează după reveniri repetate.

Logarea este pentru diagnostic, nu o operație necesară fiecărui cadru în versiunea finală.

13

Exemplu de accelerometru

Accelerometrul raportează accelerația pe trei axe, incluzând gravitația.

- getDefaultSensor(TYPE_ACCELEROMETER) caută senzorul și poate întoarce null.
- registerListener() pornește recepționarea; onSensorChanged() primește valorile; unregisterListener() oprește recepționarea.
- Înregistrăm la revenire și dezînregistrăm la pauză. Dacă senzorul lipsește, afișăm un mesaj.

Doar explicație; nu se cere în proiect.